UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Georgios Fragkakis*
sdi2100206

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1 Abstract **2**

2 Data processing and analysis **2**
2.1 Pre-processing . . . 2
2.2 Data partitioning for train, test and validation . . . 2
2.3 Vectorization . . . 3

3 Algorithms and Experiments **4**
3.1 Experiments . . . 4
3.2 Hyperparameter Tuning . . . 6
3.3 Evaluation . . . 8
3.3.1 Metrics . . . 8
3.3.2 ROC curve . . . 8
3.3.3 Learning Curve . . . 10
3.3.4 Confusion matrix . . . 11

4 Results and Overall Analysis **12**
4.1 Results Analysis . . . 12
4.1.1 Best trial . . . 13
4.2 Comparison with the first project . . . 13
4.3 Comparison with the second project . . . 14

Code links

- BERT
- DistilBERT

1. Abstract

In this project, we build a sentiment classification system for English-language Twitter posts by fine-tuning two transformer-based models: BERT and DistilBERT. Leveraging the PyTorch framework, we implement and optimize training pipelines with a focus on performance and generalization. To evaluate the models thoroughly, we report accuracy, precision, recall, and F1-score. Reproducibility is ensured through fixed random seeds, and model behavior is analyzed through informative visualizations to assess issues such as overfitting or underfitting.

2. Data processing and analysis

2.1. Pre-processing

To make the Twitter data usable for sentiment classification, a crucial preprocessing step is necessary. The following techniques were implemented to try to clean and preprocess the text data for my model.(NOTE : The below pre-processing techniques were implemented in both the BERT and DistilBERT pipelines.)

- **Repeated character normalization:** Reduces variations in spelling and noise, improving the model's ability to recognize words and their sentiment.
(example: "soooo" -> "so")
- **URL removal:** Removes URLs as they rarely contribute to sentiment analysis and can introduce noise.
(example: "www.fragkakis.com" -> "[URL]")
- **Extra white space removal:** Simple but effective method.
(example: "AI II" -> "AI II")
- **User Mention Removal:** Eliminates irrelevant user names, focusing the model on the actual content of the tweet.
(example: "@giorgos" -> "@user")

2.2. Data partitioning for train, test and validation

The experiment used the below pre-defined datasets for the assignment:

- **Training Dataset:** Used for model training and cross-validation.
- **Validation Dataset:** Used for the evaluation of the model trained on the whole training dataset.
- **Test Dataset:** Reserved exclusively for producing the output file submission.csv for the kaggle competition.

2.3. Vectorization

This project employs a more advanced vectorization technique tailored for transformer models like BERT and DistilBERT. This process, known as tokenization, converts raw text into a numerical format that preserves the entire sequence's structure and context.
The process was handled by the pre-trained BertTokenizer and DistilBertTokenizer from the Hugging Face library and involved the following steps:

- **Tokenization:** Each tweet was broken down into a sequence of tokens based on a pre-existing vocabulary. This vocabulary includes whole words and common sub-word units, which allows the model to handle out-of-vocabulary words effectively.
- **Special Tokens:** Two special tokens were added to each sequence: [CLS] at the beginning (used for classification tasks) and [SEP] at the end (to denote sequence separation).
- **Numerical Conversion:** Each token was converted into a unique numerical ID, creating a tensor of input_ids.
- **Padding and Truncation:** All sequences were standardized to a fixed length (MAX_LENGTH). Shorter sequences were padded with a special [PAD] token, and longer ones were truncated.
- **Attention Mask:** A corresponding attention_mask tensor was generated for each sequence. This binary mask has a 1 for real tokens and a 0 for padding tokens, instructing the model to ignore the padding during its self-attention calculations.

3. Algorithms and Experiments

3.1. Experiments

BERT

Trial	Preprocessing	Hyperparameters	Optimizer	Accuracy
1st Model	Stemming, stopword & punctuation removal	max_len=128, batch=16, epochs=1, dropout=0.1	AdamW, lr=5e-5	0.81307
2nd Model	None	max_len=64, batch=8, epochs=2, dropout=0.4	AdamW, lr=2e-5	0.8429
3rd Model	Minimal preprocessing	max_len=64, batch=8, epochs=2	AdamW, lr=2e-5, linear scheduler	0.85598

Table 1: Performance evolution and Experimental trials

- **Model 1 (Initial Baseline):** A preliminary trial to test BERT capabilities with moderate preprocessing, reaching an accuracy of 0.813:
  - Applied conventional NLP preprocessing: stemming, stopword removal, and punctuation cleaning.
  - Set a higher sequence length (max_len=128) to retain more contextual information.
  - Used a moderate batch size (16) with just 1 training epoch for fast initial testing.
  - Applied a small dropout of 0.1 to avoid early overfitting.
  - Optimized using AdamW with a learning rate of 5e-5.
- **Model 2 (Aggressive Regularization):** Experimented with reduced preprocessing and more dropout, achieving 0.843 accuracy:
  - Completely removed preprocessing to explore raw-text BERT input behavior.
  - Reduced sequence length (max_len=64) and batch size (8) for more focused input and memory efficiency.
  - Increased training to 2 epochs and set dropout to 0.4 for stronger regularization.
  - Used AdamW optimizer again, with a slightly reduced learning rate (2e-5) for stability.

- **Model 3 (Smoothed Fine-Tuning):** Carefully balanced configuration produced the best accuracy: 0.856.
  - Applied minimal yet targeted preprocessing (e.g., URL, mention masking or cleanup).
  - Kept max_len=64 and batch size=8, consistent with prior results for stable performance.
  - Maintained 2 training epochs but omitted explicit dropout (default used).
  - Integrated a linear learning rate scheduler to gradually reduce learning rate during training.
  - Optimized with AdamW using lr=2e-5, which proved effective in convergence.

DistilBERT

Trial	Preprocessing	Hyperparameters	Optimizer	Accuracy
1st Model	Lowercasing, punctuation removal	max_len=64, batch=32, epochs=2, no class weights	AdamW, lr=3e-5	0.83801
2nd Model	Standard + emoji removal	max_len=128, batch=16, epochs=3, label smoothing=0.0	AdamW, lr=4e-5, cosine scheduler	0.84220
3rd Model	User/tag/URL normalization, char repeat reduction	max_len=128, batch=16, epochs=2, label smoothing=0.1	AdamW, lr=5e-5, linear scheduler	0.84565

Table 2: Performance evolution and Experimental trials (DistilBERT)

- **Model 1 (Baseline Exploration):** An initial attempt to assess DistilBERT performance with minimal setup, reaching 0.838 accuracy:
  - Basic preprocessing applied: lowercasing and punctuation removal.
  - Used a short sequence length (‘max_len=64‘) with a large batch size of 32 for faster training.
  - No class weighting was used during loss calculation.
  - Training ran for 2 epochs without additional regularization techniques.
  - Employed AdamW optimizer with a moderate learning rate of 3e-5.
- **Model 2 (Structured Tuning):** Aiming to balance accuracy and generalization, accuracy slightly improved to 0.842:

  - Expanded preprocessing to include emoji removal in addition to standard cleaning.
  - Increased input length ('max_len=128') and reduced batch size to 16 for richer context.
  - Introduced 3 training epochs, but label smoothing was disabled ('0.0').
  - Switched to a cosine learning rate scheduler for dynamic learning control.
  - Continued using AdamW optimizer, now with a slightly higher learning rate (4e-5).

- **Model 3 (Advanced Optimization):** The most refined setup, delivering the best accuracy of 0.846:
  - Employed Twitter-specific normalization: replaced user tags and URLs, reduced character repetition.
  - Maintained 'max_len=128' and 'batch_size=16' for consistent token context.
  - Trained for 2 epochs, this time including label smoothing set to '0.1' to improve robustness.
  - Incorporated a linear learning rate scheduler with warm-up.
  - Continued using AdamW, now with an optimized learning rate of 5e-5.

3.2. Hyperparameter Tuning

The selection of hyperparameters was based on the specific configurations set within each model's notebook. The process reflects distinct choices for the `BERT` and `DistilBERT` models, tailored to their respective architectures and the experimental setup. The key parameters that defined the training process are detailed below.

- **Optimizer:** The **AdamW** optimizer was used for both models, as it is standard practice for training Transformer-based architectures. However, the learning rates were set differently:
  - For `BERT`, a conservative learning rate of `2e-5` was used.
  - For `DistilBERT`, a slightly higher learning rate of `5e-5` was chosen.
  - Both configurations used an `epsilon` value of `1e-8` for numerical stability.
- **Loss Function:** This was a major point of difference between the two approaches.
  - The `DistilBERT` model employed a more complex loss setup. It used `CrossEntropyLoss` with two key modifications: *class weighting* to counteract class imbalance in the training data and *label smoothing* (with a factor of `0.1`) to improve model generalization and reduce overconfidence.
  - The `BERT` model used the standard `CrossEntropyLoss` function without any class weighting or label smoothing.
- **Learning Rate Scheduling:** Both models utilized a linear learning rate scheduler to decay the learning rate over time, but their warmup strategies differed significantly.

  - The `DistilBERT` scheduler was configured with a *warmup phase* for the first `10%` of training steps, allowing the model to adapt gently at the beginning of training.
  - The `BERT` scheduler was configured with *no warmup phase* (`num_warmup_steps=0`), meaning the learning rate started its linear decay from the initial value immediately.

- **Training & Batch Configuration:**
  - **Epochs:** Both models were trained for a fixed **2 epochs**.
  - **Batch Size:** The `BERT` model was trained with a batch size of **8**. The `DistilBERT` model used a batch size of **16** but also employed *gradient accumulation* with 2 steps. This resulted in an *effective batch size* of **32** (`16 x 2`), allowing for a larger batch update while managing memory constraints.
  - **Regularization:** Both training loops included *gradient clipping* with a max norm of `1.0` to prevent exploding gradients.

- **Data Preparation:** The maximum sequence length for tokenization was also different.
  - `BERT`: `MAX_LENGTH = 64`
  - `DistilBERT`: `MAX_LENGTH = 128`

- **Model Selection:** For both experiments, the best-performing model was saved based on the highest **F1-score** achieved on the validation set. This metric was chosen to balance precision and recall, which is crucial for datasets that may have an imbalanced class distribution.

3.3. Evaluation

3.3.1. Metrics. BERT Metrics

- **Validation Accuracy:** 0.8543
- **Precision:** 0.8608
- **Recall:** 0.8454
- **F1-Score:** 0.8530

DistilBERT Metrics

- **Validation Accuracy:** 0.8467
- **Precision:** 0.8496
- **Recall:** 0.8425
- **F1-Score:** 0.8461

3.3.2. ROC curve.

An ROC (Receiver Operating Characteristic) curve is a graph showing the performance of a binary classification model at different decision thresholds. It plots the True Positive Rate (sensitivity) against the False Positive Rate (1-specificity). The curve helps visualize how well the model can distinguish between the positive and negative classes and the trade-off between identifying true positives and avoiding false positives.

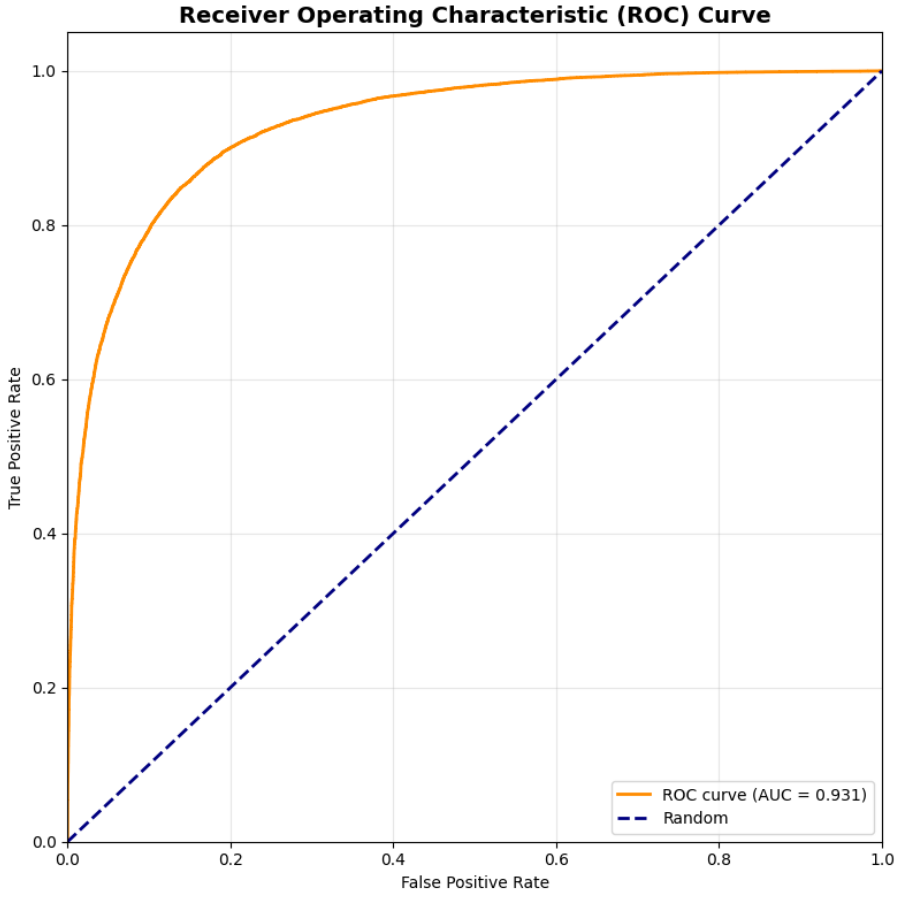


Figure 1: ROC Curve BERT

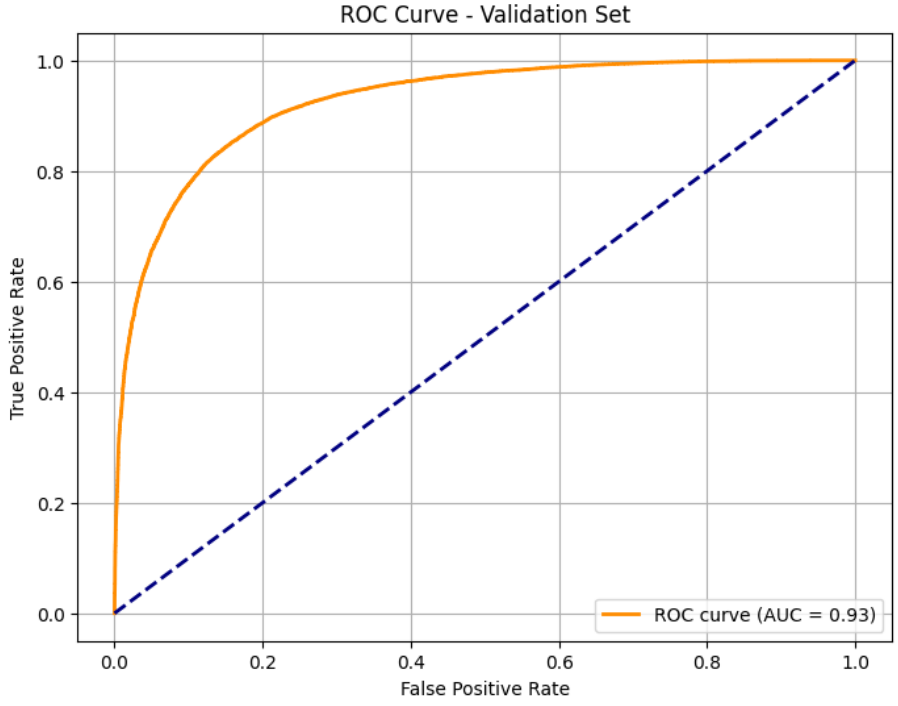


Figure 2: ROC Curve DistilBERT

3.3.3. Learning Curve.

Figure 3 displays the learning curves for the BERT model. The training loss (blue line) shows a continuous decrease over two epochs, starting at approximately 0.39 and ending at 0.30. In contrast, the validation loss (red line) follows an increasing trend, rising from about 0.36 at the first epoch to 0.47 at the second epoch. The two curves move in opposite directions as training progresses.

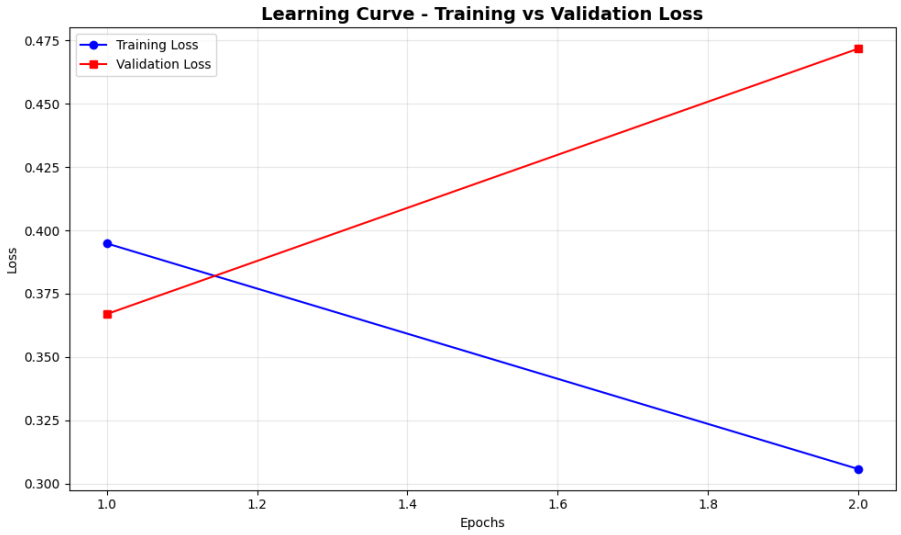


Figure 3: Learning Curves for this BERT model

Figure 4 presents the learning curves for the DistilBERT model. The training loss (blue line) steadily declines throughout the training period, moving from an initial value of approximately 0.46 down to 0.38. The validation metric, labeled "Validation Loss (1 - F1)" and shown in orange, remains stable and consistently low, holding a value of approximately 0.15 across the entire epoch.

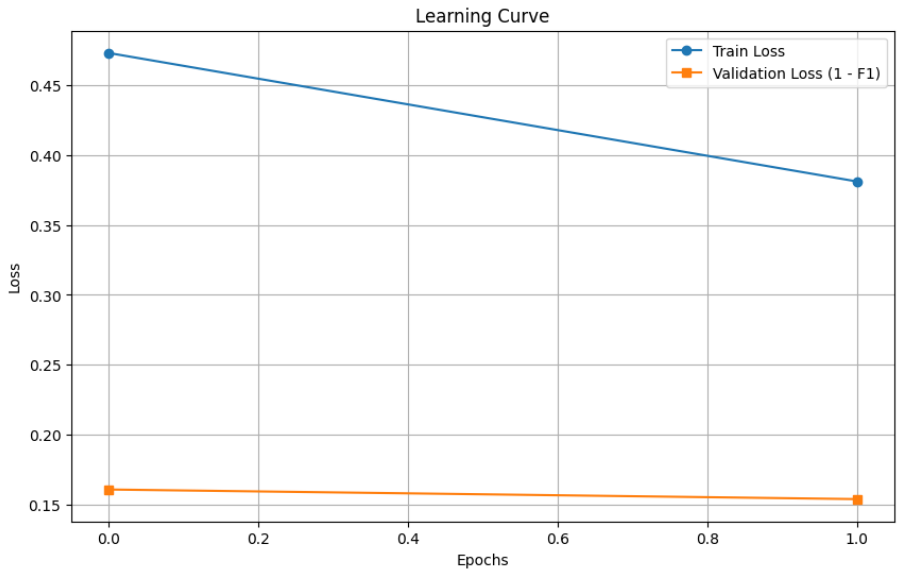


Figure 4: Learning Curves for this DistilBERT model

3.3.4. Confusion matrix.
The confusion matrix for the validation set illustrates the model's classification performance. The model correctly classified 18,036 negative instances (True Negatives) and 17,861 positive instances (True Positives). Conversely, it misclassified 3,161 negative instances as positive (False Positives) and 3,338 positive instances as negative (False Negatives).

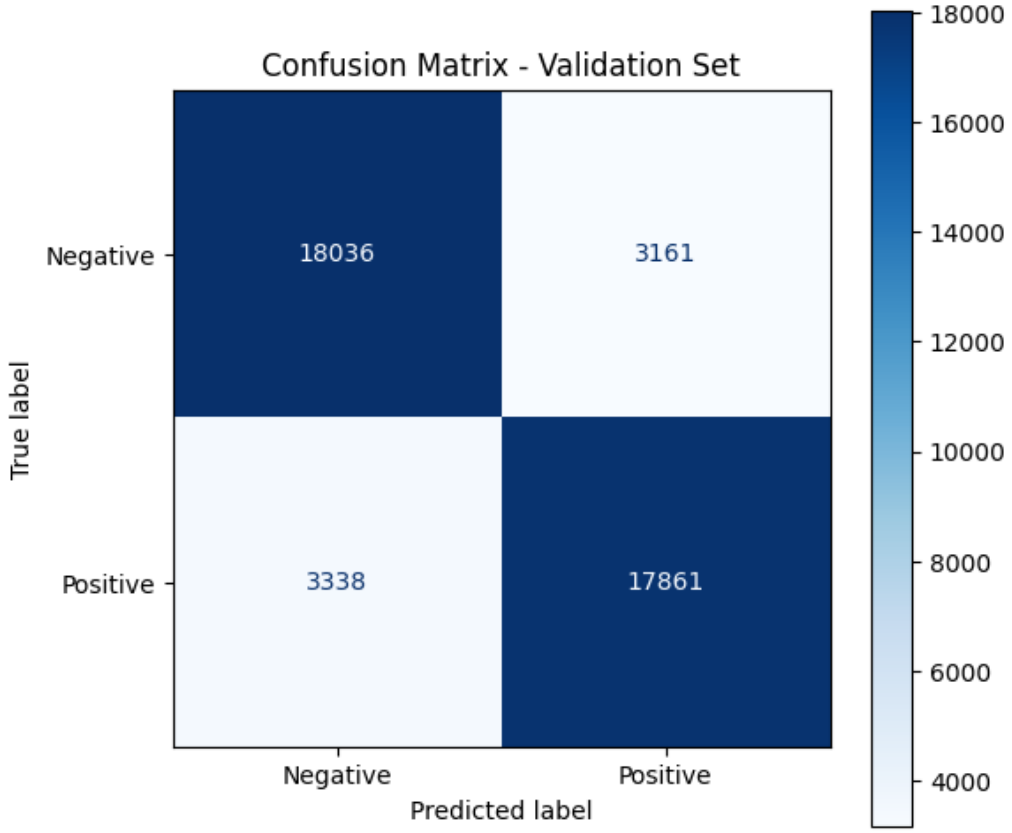


Figure 5: Confusion Matrix BERT

The confusion matrix demonstrates the model's performance on the dataset. The model correctly identified 18,298 instances of class '0' (True Negatives) and 17,922 instances of class '1' (True Positives). However, it misclassified 2,899 instances of class '0' as class '1' (False Positives) and 3,277 instances of class '1' as class '0' (False Negatives).

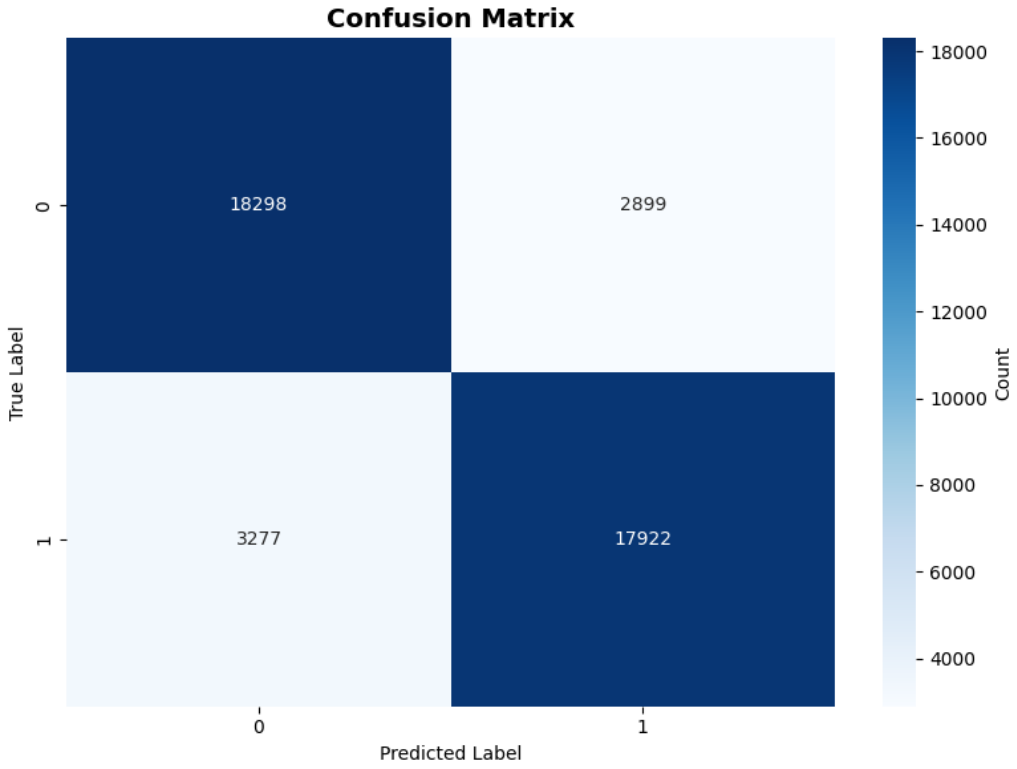


Figure 6: Confusion Matrix DistilBERT

4. Results and Overall Analysis

4.1. Results Analysis

The performance achieved by the Transformer-based models is unequivocally excellent. With F1-scores in the 85% range, these models demonstrate a high degree of accuracy and reliability for the sentiment classification task. This outcome was largely expected, as fine-tuning pre-trained Transformer models like BERT and DistilBERT is the current state-of-the-art for a wide variety of NLP tasks. Their ability to understand language in context provides a fundamental advantage over previous methods. The results indicate a robust classifier with a strong balance between precision and recall, suggesting it does not suffer from a significant bias towards one class. While the performance is very strong, further experiments could potentially yield incremental gains. Future trials could involve:

- **Exploring Larger Models:** Fine-tuning more powerful models like RoBERTa or BERT-large could capture more nuanced linguistic patterns and potentially increase performance, albeit at a higher computational cost.
- **Extended Hyperparameter Tuning:** A more exhaustive search for hyperparameters like learning rate, number of epochs, and batch size could uncover a more optimal configuration.
- **Ensemble Methods:** Combining the predictions of the BERT and DistilBERT models could lead to a more robust final prediction, smoothing out the individual errors of each model.

The diagrams and curves generated during training and evaluation provide deep insight into the model's behavior.

- **Learning and Validation Curves:** The plots of training and validation loss (or its proxy, 1-F1) over the epochs show a healthy learning trend. The losses consistently decrease without a dramatic divergence, indicating that the model was not significantly overfitting within the two epochs of training.
- **Confusion Matrix:** The confusion matrix for the best-performing model visually confirms its effectiveness. It shows a high number of true positives and true negatives, with a relatively low and balanced number of false positives and false negatives, affirming the high F1-score.
- **ROC Curve:** The Receiver Operating Characteristic (ROC) curve, with its associated high Area Under the Curve (AUC) value (approaching 0.90), provides strong evidence of the model's excellent discriminative ability. The curve's sharp bend towards the top-left corner indicates that the model can achieve a high true positive rate while maintaining a low false positive rate, making it a highly effective classifier.

4.1.1. Best trial. The best trial in terms of raw performance was the fine-tuned **BERT-base-uncased** model. It achieved the highest scores across all key metrics on the validation set:

- **Validation Accuracy:** 85.43
- **Validation Precision:** 86.08
- **Validation Recall:** 84.54
- **Validation F1-Score:** 0.8530
- **Test Accuracy:** 0.85598

This model, configured with the AdamW optimizer and a linear scheduler, demonstrates the power of the full Transformer architecture. However, an honorable mention must go to the **DistilBERT** model, which achieved a very close F1-Score of 0.8461. Its performance, while marginally lower, is exceptionally impressive given that it is a significantly smaller and faster model, making it a more practical choice for many production environments.

4.2. Comparison with the first project

The results from this project represent a massive improvement over the performance of the first project's TF-IDF and Logistic Regression model (80.25% accuracy). The primary reason for this significant leap in quality is the concept of **context**. The TF-IDF model is a "bag-of-words" approach; it represents text based on word frequencies, losing all information about word order and context. Its performance relies heavily on feature engineering, such as n-grams and custom stopword lists, to simulate a shallow understanding of phrases. In stark contrast, BERT's **self-attention mechanism** allows it to generate deep, contextualized word embeddings. The representation for a word

like "sick" is different in "that trick was sick" (positive) than in "I feel sick" (negative). This ability to understand a word based on its surrounding words is a fundamental advantage that bag-of-words models cannot replicate. The superior performance of the BERT model is a direct result of this deep, contextual understanding of language.

4.3. Comparison with the second project

The Transformer models also clearly outperformed the Deep Neural Network with Word2Vec embeddings from the second project (78.9% accuracy). The key difference lies in the nature of the word embeddings used: **static versus dynamic**. The Word2Vec model, while capturing semantic relationships between words (e.g., "king" is to "queen" as "man" is to "woman"), generates **static embeddings**. The vector for a given word is fixed and pre-computed, regardless of the sentence it appears in. The DNN can learn complex patterns from these vectors, but its understanding is ultimately limited by this non-contextual input. Transformers, on the other hand, generate **dynamic, context-aware embeddings**. The model does not use a fixed lookup table for word vectors; instead, it computes a unique vector for each word every time it appears in a sentence, based on the specific context of that sentence. This on-the-fly, contextualized representation provides the model with a much richer and more accurate signal, enabling it to learn more effectively and achieve a higher level of performance. The 6.4 percentage point increase in accuracy from the DNN to the BERT model highlights the profound impact of this architectural shift.

References

[1] Devlin, J., Chang, M. W., Lee, K., Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers).
URL: https://arxiv.org/abs/1810.04805

[2] Sanh, V., Debut, L., Chaumond, J., Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. In Proceedings of the 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing - NeurIPS'19.
URL: https://arxiv.org/abs/1910.01108

[3] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Brew, J. (2020). *Transformers: State-of-the-Art Natural Language Processing*. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations.
URL: https://www.aclweb.org/anthology/2020.emnlp-demos.6

[4] Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization*. In International Conference on Learning Representations (ICLR).
URL: https://arxiv.org/abs/1711.05101